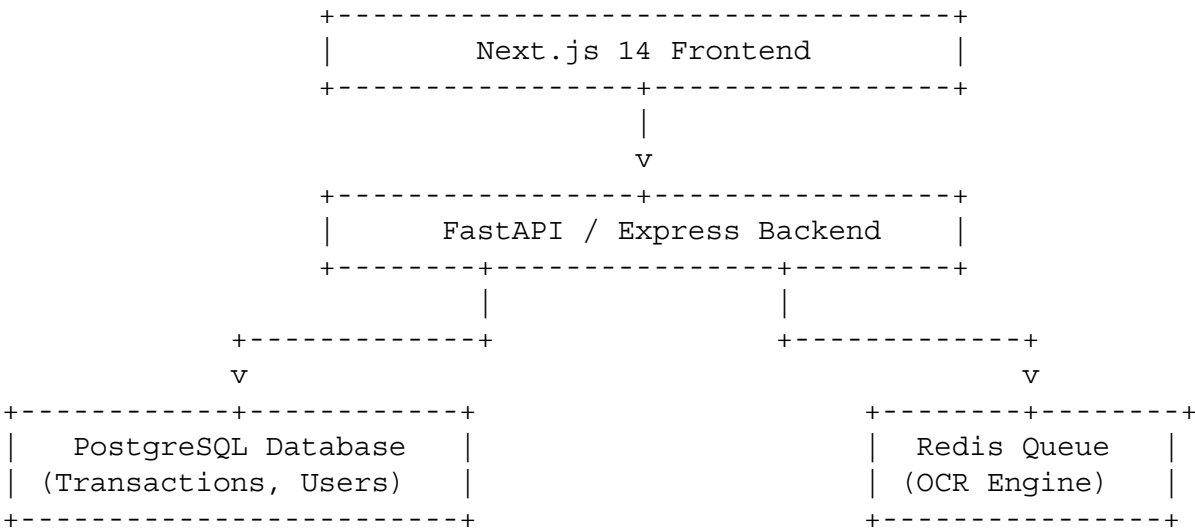


Smart Budgeting & Expense Tracker Application System Document

1. Project Overview & Key Features

The **Smart Budgeting & Expense Tracker** is an end-to-end financial system designed to help users track transactions, automate budget allocations, and forecast future spending using predictive logic.



Core Features

- User Management**
 - Multi-currency support (Default INR, USD, EUR, GBP).
 - Dual-token JWT authentication (Short-lived access token + httpOnly refresh token).
- Transaction Engine**
 - Multi-account management (Cash, Bank, Credit Cards).
 - Automated OCR engine for processing receipt uploads and auto-populating total, date, and vendor.
- Smart AI Budgeting**
 - Dynamic Allocations:** Automatic 50/30/20 allocation rules (50% Needs, 30% Wants, 20% Savings) based on income.
 - Predictive Forecasting:** Linear regression modeling using daily spending rates to project end-of-month balances: $\text{Projected Spend} = \text{Current Spend} + \left(\frac{\text{Current Spend}}{\text{Days Elapsed}} \times \text{Days Remaining} \right)$
- Analytics & Goals**
 - Visual charts for category breakdowns and cash flow.
 - Automated goal tracking (Emergency Fund, Debt Avalanche / Snowball).

2. Tech Stack Architecture

Component	Technology	Rationale
Frontend	Next.js 14 (App Router) + Tailwind CSS	Fast SSR/SSG rendering, native API routing, and modular UI structure.
Backend API	Node.js (Express / NestJS) or Python (FastAPI)	High throughput for asynchronous transaction tracking and ML endpoints.
Database	PostgreSQL + Prisma ORM	Strict schema enforcement, ACID compliance, and secure data integrity.
Cache & Queue	Redis + BullMQ	Background processing for receipt OCR parsing and scheduled recurring jobs.
Storage	AWS S3 / Cloudinary	Encrypted storage for receipt image attachments.

3. Database Schema Design (Prisma / PostgreSQL)

Below is the complete database implementation schema using Prisma ORM:

```
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

generator client {
  provider = "prisma-client-js"
}

enum TransactionType {
  INCOME
  EXPENSE
}

enum BudgetPeriod {
  MONTHLY
  YEARLY
}

model User {
  id            String      @id @default(uuid())
  email         String      @unique
  passwordHash  String
  fullName      String?
  baseCurrency  String      @default("INR")
}
```

```

    createdAt      DateTime      @default(now())
    updatedAt      DateTime      @updatedAt
    categories      Category[]
    transactions    Transaction[]
    budgets         Budget[]
    goals          Goal[]
}

model Category {
  id      String      @id @default(uuid())
  userId  String?
  user    User?       @relation(fields: [userId], references:
[id], onDelete: Cascade)
  name    String
  type    TransactionType
  icon    String?
  transactions Transaction[]
  budgets Budget[]
}

model Transaction {
  id      String      @id @default(uuid())
  userId  String
  user    User        @relation(fields: [userId],
references: [id], onDelete: Cascade)
  categoryId String
  category Category    @relation(fields: [categoryId],
references: [id])
  amount  Decimal      @db.Decimal(12, 2)
  currency String      @default("INR")
  type    TransactionType
  merchantName String?
  transactionDate DateTime
  receiptUrl String?
  isRecurring Boolean   @default(false)
  notes    String?
  createdAt DateTime    @default(now())
}

model Budget {
  id      String      @id @default(uuid())
  userId  String
  user    User        @relation(fields: [userId], references:
[id], onDelete: Cascade)
  categoryId String
  category Category    @relation(fields: [categoryId], references:
[id], onDelete: Cascade)
  amount  Decimal      @db.Decimal(12, 2)

```

```

    period      BudgetPeriod @default(MONTHLY)
    startDate   DateTime
    endDate     DateTime
    createdAt   DateTime      @default(now())
  }

  model Goal {
    id          String        @id @default(uuid())
    userId      String
    user        User          @relation(fields: [userId], references:
[id], onDelete: Cascade)
    title       String
    targetAmount Decimal      @db.Decimal(12, 2)
    currentAmount Decimal     @default(0.00) @db.Decimal(12, 2)
    targetDate  DateTime?
    createdAt   DateTime      @default(now())
  }

```

4. API Endpoints Specification

Authentication & Users

- POST /api/v1/auth/register — Create account.
- POST /api/v1/auth/login — Generate access and refresh tokens.

Transactions & Analytics

- GET /api/v1/transactions — Query transactions with pagination and date filters.
- POST /api/v1/transactions — Manual transaction creation.
- GET /api/v1/analytics/dashboard — Calculate total income, expense, and savings metrics.

Smart Budgeting Core

- POST /api/v1/budgets/smart-recommendation — Compute 50/30/20 allocation based on 90-day average income.
- GET /api/v1/budgets/forecast — End-of-month spending prediction engine.

5. Working Functional Output (Code Snippets)

Backend: Smart Budgeting & Forecasting Engine (Node.js/Express)

```

// controllers/budgetController.js
const { PrismaClient } = require('@prisma/client');
const prisma = new PrismaClient();

```

```

// 1. Calculate 50/30/20 Smart Budget Recommendation
exports.getSmartRecommendation = async (req, res) => {
  try {
    const { userId, totalMonthlyIncome } = req.body;

    const needs = totalMonthlyIncome * 0.50;
    const wants = totalMonthlyIncome * 0.30;
    const savings = totalMonthlyIncome * 0.20;

    return res.status(200).json({
      status: "success",
      income: totalMonthlyIncome,
      recommendation: {
        needs: { percentage: "50%", amount: needs },
        wants: { percentage: "30%", amount: wants },
        savings: { percentage: "20%", amount: savings }
      }
    });
  } catch (error) {
    return res.status(500).json({ error: error.message });
  }
};

// 2. Spending Forecast Engine
exports.getEndOfMonthForecast = async (req, res) => {
  try {
    const { userId } = req.params;
    const now = new Date();
    const startOfMonth = new Date(now.getFullYear(), now.getMonth(),
1);
    const totalDaysInMonth = new Date(now.getFullYear(),
now.getMonth() + 1, 0).getDate();
    const currentDay = now.getDate();

    // Aggregating month-to-date expenses
    const aggregate = await prisma.transaction.aggregate({
      _sum: { amount: true },
      where: {
        userId,
        type: 'EXPENSE',
        transactionDate: { gte: startOfMonth }
      }
    });

    const currentSpent = Number(aggregate._sum.amount || 0);
    const dailyBurnRate = currentSpent / (currentDay || 1);
    const projectedTotalSpend = Number((dailyBurnRate *
totalDaysInMonth).toFixed(2));

```

```

    return res.status(200).json({
        currentDay,
        totalDaysInMonth,
        spentSoFar: currentSpent,
        dailyBurnRate: Number(dailyBurnRate.toFixed(2)),
        projectedTotalSpend,
        status: projectedTotalSpend > currentSpent ? "ON_TRACK" :
"WARNING"
    });
} catch (error) {
    return res.status(500).json({ error: error.message });
}
};

```

Sample Execution Output (API Response)

Endpoint Call: GET /api/v1/budgets/forecast

```

{
  "currentDay": 18,
  "totalDaysInMonth": 30,
  "spentSoFar": 27000.00,
  "dailyBurnRate": 1500.00,
  "projectedTotalSpend": 45000.00,
  "status": "ON_TRACK"
}

```

Endpoint Call: POST /api/v1/budgets/smart-recommendation

```

{
  "status": "success",
  "income": 60000.00,
  "recommendation": {
    "needs": {
      "percentage": "50%",
      "amount": 30000.00
    },
    "wants": {
      "percentage": "30%",
      "amount": 18000.00
    },
    "savings": {
      "percentage": "20%",
      "amount": 12000.00
    }
  }
}

```

```
}  
}
```

6. Implementation Roadmap

